

Sistema de emergencia

- Tomas Jaures (21908413724)

Diseño del sistema

Para el sistema, se ha implementado la estructura de la siguiente carpeta:

```
system
├── src
│   ├── data
│   ├── models
│   ├── services
│   ├── structures
│   └── utils
```

- `data` : Es donde va a almacenada la informacion, como datasets
- `models` : Clases especificas para el sistema de emergencias
- `services` : Clases estaticas con el objetivo de ofrecer un servicio
- `structures` : Estructuras de datos como Heap, Grafos, Etc.
- `utils` : Clases auxiliares para diseño del ejercicio

Gracias a las clases de `utils` podemos cargar y crear los Datasets para el ejercicio dado, con `structures` podemos apoyarnos de estructuras como Heap, PriorityQueue o HashTable que nos seran de ayuda no solo para trabajar con las clases como Graph y Nodes, si no que estan implementadas de tal manera que pueden ser de ayuda para cualquier ejercicio. Ademas, gracias a los servicios tales como `Arrays` o `SearchAlgorithms` podemos ordenar arreglos o ejecutar busquedas dentro de nuestro grafo. Es con estas herramientas que podemos comenzar a usar los models creados, donde se representan los datos como los incidentes, centros y la red vial que logra almacenar y asociar los centros y incidentes a los nodos contenidos en el grafo. Finalmente, `EmergencySystem` centraliza toda la logica del sistema, para tener un entorno codificado para gestionar cualquier tipo de emergencias, dando informacion clave como la ruta mas rapida a un incidentes, acceder a informacion de los incidentes de manera veloz, obtener estadisticas en base a diversos factores, etc.

A continuacion, se dara el contexto de la mayoria de clases utilizadas y su aplicacion.

Modelos

1. TDA Incident (Incidente)

Descripción: Representa un evento o emergencia registrado en el sistema. Posee atributos que definen su estado, ubicación, tipo, gravedad y una prioridad dinámica que se calcula de forma automática con base en el tiempo transcurrido desde que ocurrió el incidente.

Esta además, se puede apoyar de otras ENUM's tales como `IncidentStates` o `IncidentTypes` para conocer a mejor detalle que información es posible almacenar.

OPERACIONES Y MÉTODOS:

- `recalculatePriority() -> Float`: Calcula y actualiza la prioridad del incidente multiplicando su nivel de gravedad (`severity`) por el tiempo total transcurrido desde su creación en minutos (convertido a días). Retorna el valor redondeado a un decimal.
- `toString() -> String`: Retorna una cadena de texto resumida con los datos básicos del incidente (ID, ubicación, estado y prioridad).
- `getData() -> String`: Retorna un reporte textual detallado que incluye la ruta más cercana (`closestPath`) y el centro de emergencia asignado (`closestCenter`).

Comportamiento/Operadores:

- **Igualdad (`__eq__`)**: Permite comparar si dos incidentes son iguales verificando únicamente su identificador (`id`), ya sea contra otra instancia de `Incident` o contra una cadena de texto.
- **Comparación (`__gt__`)**: Sobrescribe el operador `>` delegándolo al comparador activo (por defecto, ordenamiento por prioridad), facilitando su inserción en colas de prioridad o estructuras de ordenamiento. Este método está directamente relacionado a los comparadores posibles de la clase.
- `comparatorByPriority(a, b) -> Boolean`: Método estático que compara dos incidentes y retorna `True` si el incidente `a` tiene mayor prioridad que el incidente `b`.
- `comparatorByDatetime(a, b) -> Boolean`: Método estático que compara dos incidentes basándose en su antigüedad, retornando `True` si el incidente `a` ocurrió antes que `b`.

2. TDA EmergencyCenter (Centro de Emergencia)

Descripción:

Entidad que representa una central o base de operaciones de emergencia dentro de una zona o locación del sistema. Su propósito es servir como origen para el despacho de ayuda hacia los incidentes.

OPERACIONES Y MÉTODOS:

- `getData() -> String`: Retorna una representación formateada en texto con los atributos clave de la central: su ID, nombre comercial o institucional y su ubicación exacta dentro del mapa.
- **Comportamiento/Operadores:**
- **Igualdad (`__eq__`)**: Define el criterio de identidad del centro validando la coincidencia de su propiedad `id` (acepta comparaciones con objetos del mismo tipo o strings).
- **Mapeo (`__hash__`)**: Retorna el valor hash basado en su `id`, habilitando la posibilidad de almacenar centros de emergencia de forma directa y eficiente dentro de conjuntos o tablas hash.

3. TDA RoadNetwork (Red Vial / Red de Caminos)

Descripción: Estructura compuesta de alto nivel que modela la infraestructura vial de una ciudad o zona geográfica. Utiliza de manera interna un `Graph` para gestionar la topología de la red y una `HashTable` para indexar y buscar de manera eficiente los nodos de los caminos mediante sus IDs.

OPERACIONES:

- `addNode(id) -> Void` : Crea un nodo de la red vial con un identificador único, registrándolo en la estructura del grafo principal y agregándolo simultáneamente a la tabla hash interna de búsqueda rápida.
- `addNodes(ids: List) -> Void` : Itera e inserta múltiples identificadores de nodos en la red vial de forma masiva.
- `addEdge(originId, targetId, weight) -> Void` : Recupera los nodos correspondientes desde la tabla hash y establece un camino o calle con un peso asociado (por ejemplo, distancia o tiempo). Lanza un error (`ValueError`) si alguno de los extremos no existe previamente en la red.
- `addEdges(edges: List[Tuple]) -> Void` : Recibe una lista de tuplas con la estructura (`origen, destino, peso`) para trazar y dar de alta múltiples aristas o tramos de caminos consecutivamente.

Estructuras de datos

Para implementar la correcta implementacion de Sistema de emergencias, se realizaron diversas estructuras de datos. La gran cualidad de estas mismas, es que son completamente genericas, es decir, que no son especificas de este ejercicio, si no mas bien que pueden ser usadas en cualquier contexto, llegando a implementarse incluso una dentro de otra.

1. TDA HashTable

Descripción: Estructura de datos asociativa que almacena pares clave-valor utilizando una función hash polinomial para indexar y listas dinámicas (buckets) para resolver colisiones.

OPERACIONES:

- `insert(key, value) -> Boolean` : Inserta un par clave-valor. Retorna `True` si se insertó con éxito o `False` si la clave ya existía.
- `search(key) -> Value | None` : Busca una clave en la tabla y retorna su valor asociado. Si no existe, retorna `None` .
- `update(key, value) -> Boolean` : Modifica el valor asociado a una clave existente. Retorna `True` si se actualizó correctamente o `False` si la clave no fue encontrada.
- `remove(key) -> Value | None` : Elimina una clave de la tabla y retorna su valor. Si la clave no existe, retorna `None` .
- `loadFactor() -> Float` : Calcula y retorna el factor de carga actual de la tabla (elementos/capacidad).

- `getKeys() -> List` : Retorna una lista con todas las claves presentes en la tabla.
- `getStats() -> String` : Retorna una cadena de texto con las estadísticas de rendimiento de la tabla (factor de carga, colisiones, buckets ocupados, etc.).

2. TDA Heap

Descripción: Estructura de árbol binario completo basada en arreglos que cumple la propiedad de Max-Heap.

OPERACIONES:

- `insert(value) -> Boolean` : Inserta un elemento al final del arreglo y lo reubica hacia arriba mediante un *sift-up* para mantener la propiedad de Max-Heap.
- `extract() -> Value | None` : Extrae y retorna el elemento con la máxima prioridad (raíz). Reubica el último elemento a la raíz y realiza un *sift-down*.
- `updateElement(element) -> Boolean` : Busca y actualiza un elemento dentro del Heap, reordenando el árbol hacia arriba o hacia abajo según sea necesario. Retorna `True` si se actualizó.
- `topK(k) -> List` : Retorna de manera ordenada los k elementos con mayor prioridad del heap sin destruir la estructura original.
- `buildHeap(arr: List) -> Void` : Toma una lista desordenada y la transforma internamente en un Heap válido utilizando el algoritmo de *heapify* descendente.

3. TDA PriorityQueue

Descripción: Estructura lineal adaptada cuya lógica interna está respaldada por un Heap, donde los elementos se procesan y extraen en función de su nivel de prioridad.

OPERACIONES:

- `push(element) -> Boolean` : Añade un elemento a la cola de prioridad respetando su orden respectivo.
- `pop() -> Element | None` : Remueve y retorna el elemento con la mayor prioridad de la cola.
- `peek() -> Element` : Retorna el elemento en el frente (mayor prioridad) sin removerlo de la cola.
- `isEmpty() -> Boolean` : Retorna `True` si la cola está vacía o `False` en caso contrario.
- `getSize() -> Integer` : Retorna la cantidad total de elementos actualmente en la cola.
- `pushAll(arr: List) -> Void` : Construye eficientemente la cola a partir de una lista completa de elementos usando *buildHeap*.
- `top(k) -> List` : Obtiene los mejores k elementos prioritarios de la cola.
- `updateElement(element) -> Void` : Actualiza dinámicamente el valor/prioridad de un elemento dentro de la cola.

4. TDA Graph

Descripción: Colección de nodos (vértices) interconectados por aristas que pueden poseer pesos numéricos, orientados al modelamiento de redes o caminos.

OPERACIONES:

- `addNode(node)` : Registra un nuevo nodo único en el conjunto del grafo.
- `addNodes(nodes: List)` : Inserta de manera masiva múltiples nodos al grafo.
- `addEdge(origin, target, weight, addressed)` : Crea una conexión (arista) entre un nodo origen y un nodo destino con un peso específico. Si `addressed` es `False` , la arista se crea de forma bidireccional.
- `show() -> Void` : Imprime en consola la representación visual del grafo detallando cada nodo con sus respectivas adyacencias y pesos.

5. TDA Node

- **Descripción:** Componente fundamental del grafo que almacena un dato genérico, mantiene una lista de sus conexiones directas y posee atributos especiales (`currentWeight` y `parent`) optimizados para algoritmos de rutas mínimas como Dijkstra.
- **Operaciones / Comportamiento:**
 - `addAdjacency(node, weight)` : Añade una tupla que representa una conexión dirigida hacia otro nodo junto al peso de la arista.
 - `setParent(parent)` : Asigna el nodo predecesor en la ruta óptima calculada.
 - **Comparadores (`<` , `>`):** Modifican los operadores lógicos `>` y `<` para que un nodo sea considerado "menor" cuando tiene un `currentWeight` mayor, permitiendo que un Max-Heap se comporte como un **Min-Heap** idóneo para Dijkstra.

TDA EmergencySystem (Sistema de Emergencias)

Descripción: Componente de orquestación y lógica de negocio de alto nivel. Centraliza la infraestructura vial (`RoadNetwork`), la gestión indexada de incidentes y centros (mediante `HashTable`) y la priorización de emergencias en tiempo real (mediante `PriorityQueue`).

OPERACIONES DE GESTIÓN DE ENTIDADES:

- `insertCenter(center)` : Registra un centro de emergencias en el sistema de manera indexada y asocia su ubicación física al nodo correspondiente en la red vial. Lanza un error si ya existe.
- `insertIncident(incident)` : Da de alta un nuevo incidente de forma simultánea en la estructura de búsqueda rápida (tabla hash) y en el planificador de criticidad (cola de prioridad).
- `searchIncident(id) -> Incident | None` : Recupera en tiempo constante aproximado un incidente a partir de su identificador único.
- `updateIncidentState(id, state)` : Modifica el estado actual del incidente (ej. en curso, resuelto) dentro del registro principal.

- `removeIncident(id) -> Incident | None` : Elimina un incidente del registro hash y devuelve su información asociada.

OPERACIONES DE PRIORIZACIÓN Y ANÁLISIS:

- `getMostUrgentIncident() -> Incident` : Consulta la raíz de la cola de prioridad para conocer el incidente más crítico de forma inmediata ($O(1)$).
- `getKIncidentsMostCritic(k) -> List` : Extrae un subconjunto ordenado con las k emergencias que requieren mayor atención.
- `updateIncidentPriority(id, newPriority)` : Modifica el nivel de urgencia de un incidente y reestructura internamente la cola de prioridad para mantener la integridad del Heap.

OPERACIONES DE ORDENAMIENTO DINÁMICO:

- `getIncidentsByTime() -> List` : Extrae el arreglo interno de incidentes, conmuta temporalmente el comparador a orden cronológico y ejecuta un algoritmo `heapSort` para devolver las emergencias de menor a mayor antigüedad.
- `getIncidentsByPriority() -> List` : Exporta una lista completa con la totalidad de los incidentes ordenados por nivel de urgencia actual.

OPERACIONES DE ENRUTAMIENTO Y OPTIMIZACIÓN:

- `mostEfficientRouteFromCenterToIncident(center, incident) -> Tuple` : Ejecuta el algoritmo de Dijkstra para trazar la ruta de costo mínimo (distancia/tiempo) desde un centro específico hasta el lugar del incidente.
- `mostEfficientRouteToIncidentByBFS(incident) -> List` : Realiza una exploración en anchura (BFS) partiendo desde la locación del incidente hacia atrás para ubicar el centro de emergencia más cercano en cantidad de calles.
- `mostEfficientRouteToIncidentByDijkstra(incident) -> List` : Aplica Dijkstra en sentido inverso partiendo desde el incidente para determinar cuál es el centro de respuesta óptimo evaluando los pesos acumulados de la red.

1. TDA SearchAlgorithms (Algoritmos de Búsqueda en Grafos)

- **Descripción:** Componente estático que provee algoritmos fundamentales de exploración y optimización sobre estructuras de grafos. Permite encontrar caminos y nodos que cumplan con criterios específicos mediante recorridos sistemáticos.

- **Operaciones:**

- `bfs(startNode, condition) -> Tuple | None`: Ejecuta una búsqueda en anchura (*Breadth-First Search*) a partir de un nodo inicial. Recorre el grafo de forma nivel por nivel usando una cola (`deque`) y una tabla hash para registrar los nodos visitados. Si un nodo cumple con la función de evaluación `condition`, reconstruye y retorna el nodo objetivo, la secuencia del camino óptimo en número de aristas y la lista de nodos explorados. Si no hay coincidencias, retorna `None`.
- `dijkstra(startNode, condition) -> Tuple`: Implementa el algoritmo de Dijkstra para hallar la ruta más corta en un grafo con aristas de peso positivo. Utiliza una cola de prioridad para evaluar primero los caminos de menor costo acumulado y gestiona las estructuras de precedencia mediante tablas hash. Al finalizar, limpia y reinicia los pesos de los nodos modificados a infinito (`inf`), retornando el nodo destino encontrado, el camino recorrido, los nodos visitados y el costo o distancia total acumulada.

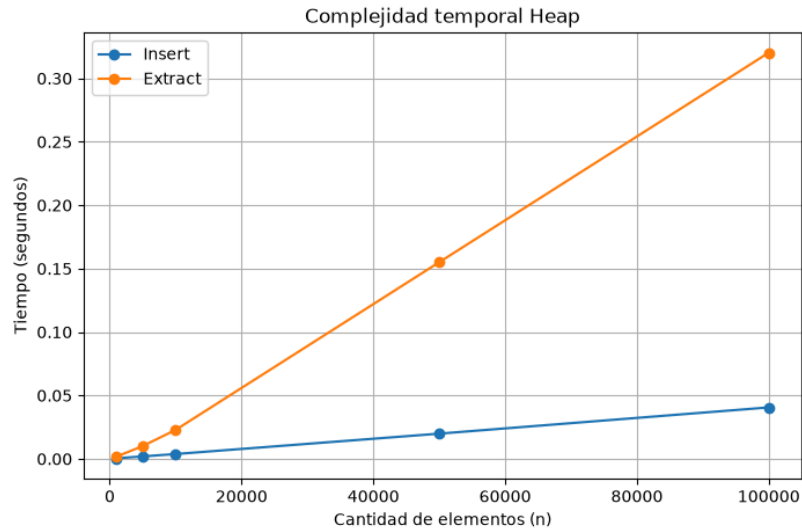
2. TDA Arrays (Algoritmos de Ordenamiento)

- **Descripción:** Biblioteca de funciones estáticas especializada en el ordenamiento y reestructuración de arreglos lineales o listas de elementos mutables.
- **Operaciones:**
 - `heapSort(arr: List) -> List`: Ordena una lista de elementos de menor a mayor utilizando la propiedad del TDA Heap. Construye un árbol de prioridad con el arreglo original, extrae sucesivamente el elemento máximo y construye una lista invertida para entregar el resultado ordenado ascendentemente.
 - `quickSort(array: List, low, high) -> List`: Ordena una lista *in-place* aplicando el principio de "divide y vencerás". Selecciona de forma recursiva un elemento como pivote para segmentar el arreglo en sub-listas de elementos menores y mayores, optimizando el uso de memoria al no requerir arreglos auxiliares.
 - `_partition(array, low, high) -> Integer`: Sub-operación privada utilizada por `quickSort`. Ubica los elementos menores o iguales al pivote a su izquierda y los mayores a su derecha, situando finalmente al pivote en su posición indexada correcta y retornando dicho índice.

ANÁLISIS EXPERIMENTAL

Heap

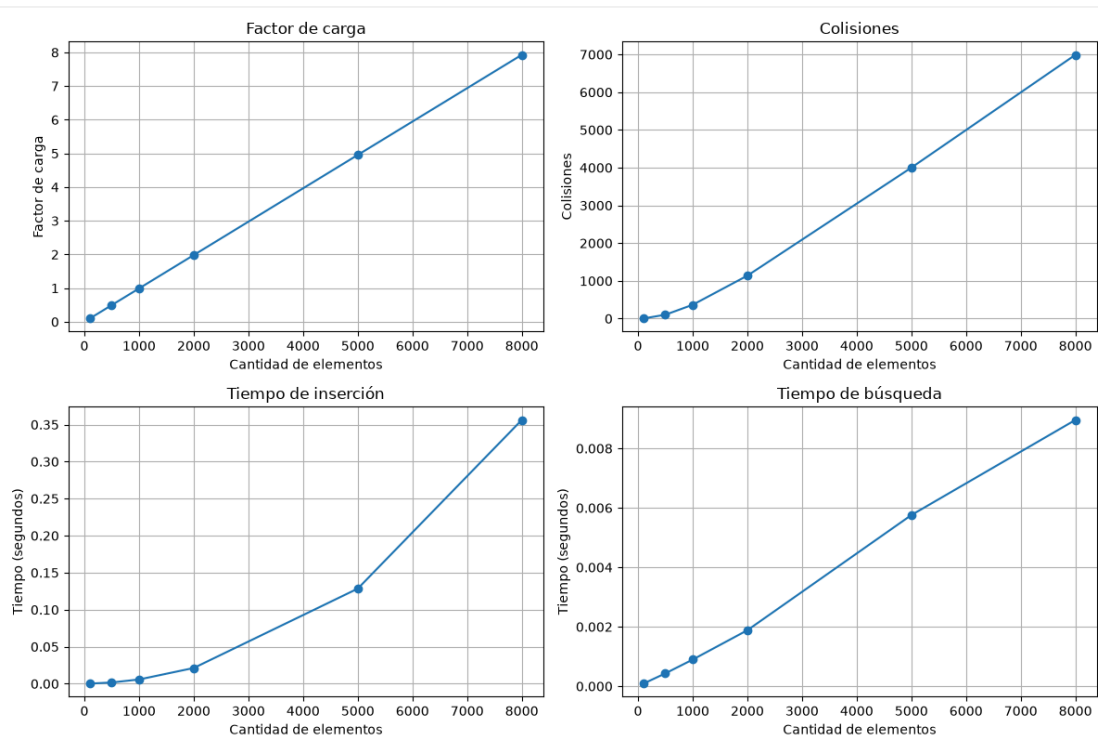
Se realizó un análisis experimental del comportamiento del Heap mediante la medición del tiempo de ejecución de las operaciones principales de inserción y extracción.



Los resultados muestran que, al aumentar la cantidad de elementos almacenados, el tiempo de ejecución incrementa de forma lentamente progresiva. Esto coincide con el comportamiento esperado de un Heap binario, donde tanto la inserción como la extracción poseen una complejidad temporal de $O(\log n)$ por operación, debido a que ambas operaciones recorren como máximo la altura del árbol.

Hashing: Colisiones, Factor de carga y Rendimiento

Se evaluó el comportamiento de la tabla hash considerando la cantidad de colisiones generadas, el factor de carga y el tiempo de ejecución de las operaciones de inserción y búsqueda.

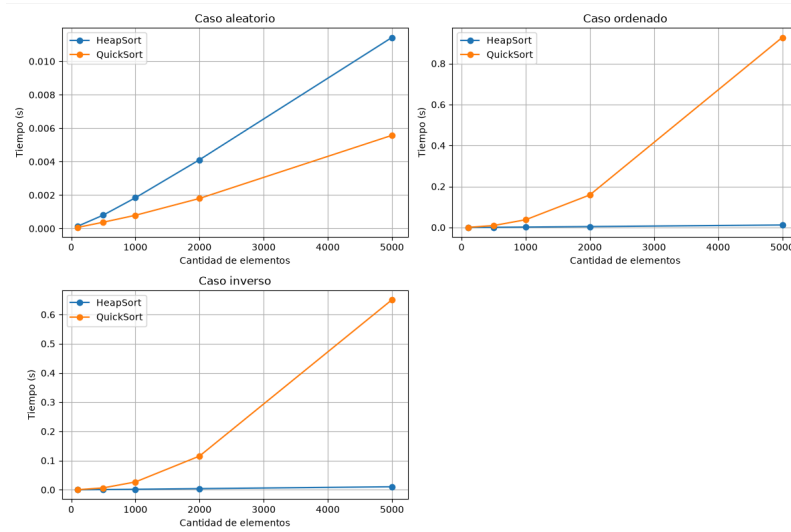


A medida que aumenta la cantidad de elementos insertados, el factor de carga incrementa debido a que existe una mayor ocupación de los buckets disponibles. Como consecuencia, aumenta la probabilidad de que múltiples claves sean asignadas a una misma posición, generando más colisiones.

El rendimiento de la estructura se mantiene eficiente mientras el factor de carga permanece bajo, y ya que la resolución de colisiones mediante listas enlazadas permite mantener tiempos extremadamente bajos (Se puede observar gracias al tiempo tan mínimo realizado en la búsqueda). Sin embargo, cuando aumenta considerablemente la cantidad de elementos, los buckets contienen más elementos y las operaciones pueden acercarse al peor caso de $O(n)$, que aun así es conveniente

Sorting: Comparación entre algoritmos

Se comparó el rendimiento de los algoritmos **HeapSort** y **QuickSort** utilizando arreglos con distintos tamaños y órdenes iniciales.



Los resultados muestran diferencias en el comportamiento de ambos algoritmos según la distribución inicial de los datos. HeapSort mantiene un rendimiento estable debido a que siempre conserva una complejidad de $O(n \log n)$, independiente del orden del arreglo.

Por otro lado, QuickSort presenta un mejor rendimiento promedio únicamente cuando el arreglo está desordenado, esto debido a la eficiencia de sus particiones y su pivote, alcanzando una complejidad esperada de $O(n \log n)$. Sin embargo, debido a la selección del pivote como último elemento, puede presentar una degradación en casos donde el arreglo ya se encuentra ordenado o inversamente ordenado, alcanzando una complejidad de $O(n^2)$ quedando en una peor posición ante HeapSort.

En conclusión, HeapSort ofrece mayor estabilidad en el peor caso, mientras que QuickSort puede obtener mejores tiempos en escenarios promedio debido a una menor cantidad de operaciones internas.